



AIML UNIT III Notes

Computer Science SL (Meenakshi College of Engineering)



Scan to open on Studocu

UNIT III SUPERVISED LEARNING

Introduction to machine learning – Linear Regression Models: Least squares, single & multiple variables, Bayesian linear regression, gradient descent, Linear Classification Models: Discriminant function – Probabilistic discriminative model - Logistic regression, Probabilistic generative model – Naive Bayes, Maximum margin classifier – Support vector machine, Decision Tree, Random forests

INTRODUCTION

- Machine learning is programming computers to optimize a performance criterion using example data or past experience . We have a model defined up to some parameters, and learning is the execution of a computer program to optimize the parameters of the model using the training data or past experience. The model may be predictive to make predictions in the future, or descriptive to gain knowledge from data.
- The field of study known as machine learning is concerned with the question of how to construct computer programs that automatically improve with experience.

Machine learning is a subfield of artificial intelligence that involves the development of algorithms and statistical models that enable computers to improve their performance in tasks through experience. These algorithms and models are designed to learn from data and make predictions or decisions without explicit instructions. There are several types of machine learning, including supervised learning, unsupervised learning, and reinforcement learning. Supervised learning involves training a model on labeled data, while unsupervised learning involves training a model on unlabeled data. Reinforcement learning involves training a model through trial and error. Machine learning is used in a wide variety of applications, including image and speech recognition, natural language processing, and recommender systems.

Definition of learning: A computer program is said to *learn* from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks T, as measured by P , improves with experience E.

Examples

- Handwriting recognition learning problem
 - Task T : Recognizing and classifying handwritten words within images
 - Performance P : Percent of words correctly classified
 - Training experience E : A dataset of handwritten words with given classifications
- A robot driving learning problem
 - Task T : Driving on highways using vision sensors
 - Performance P : Average distance traveled before an error
 - Training experience E : A sequence of images and steering commands recorded while observing a human driver

Definition: A computer program which learns from experience is called a machine learning program or simply a learning program .

Classification of Machine Learning

Machine learning implementations are classified into four major categories, depending on the nature of the learning “signal” or “response” available to a learning system which are as follows:

A. Supervised learning:

Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs. The given data is labeled . Both *classification* and *regression* problems are supervised learning problems .

- Example — Consider the following data regarding patients entering a clinic . The data consists of the gender and age of the patients and each patient is labeled as “healthy” or “sick”.

gender	age	label
M	48	sick
M	67	sick
F	53	healthy
M	49	sick
F	32	healthy
M	34	healthy
M	21	healthy

B. Unsupervised learning:

Unsupervised learning is a type of machine learning algorithm used to draw inferences from datasets consisting of input data without labeled responses. In unsupervised learning algorithms, classification or categorization is not included in the observations. Example: Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients.

gender	age
M	48
M	67
F	53
M	49
F	34
M	21

As a kind of learning, it resembles the methods humans use to figure out that certain objects or events are from the same class, such as by observing the degree of similarity between objects.

Some recommendation systems that you find on the web in the form of marketing automation are based on this type of learning.

C. Reinforcement learning:

Reinforcement learning is the problem of getting an agent to act in the world so as to maximize its rewards.

A learner is not told what actions to take as in most forms of machine learning but instead must discover which actions yield the most reward by trying them. For example — Consider teaching a dog a new trick: we cannot tell him what to do, what not to do, but we can reward/punish it if it does the right/wrong thing.

When watching the video, notice how the program is initially clumsy and unskilled but steadily improves with training until it becomes a champion.

D. Semi-supervised learning:

Where an incomplete training signal is given: a training set with some (often many) of the target outputs missing. There is a special case of this principle known as Transduction where the entire set of problem instances is known at learning time, except that part of the targets are missing. Semi-supervised learning is an approach to machine learning that combines small labeled data with a large amount of unlabeled data during training. Semi-supervised learning falls between unsupervised learning and supervised learning.

Categorizing based on required Output

Another categorization of machine learning tasks arises when one considers the desired output of a machine-learned system:

1. **Classification:** When inputs are divided into two or more classes, the learner must produce a model that assigns unseen inputs to one or more (multi-label classification) of these classes. This is typically tackled in a supervised way. Spam filtering is an example of classification, where the inputs are email (or other) messages and the classes are “spam” and “not spam”.
2. **Regression:** Which is also a supervised problem, A case when the outputs are continuous rather than discrete.
3. **Clustering:** When a set of inputs is to be divided into groups. Unlike in classification, the groups are not known beforehand, making this typically an unsupervised task.

Machine Learning comes into the picture when problems cannot be solved using typical approaches. ML algorithms combined with new computing technologies promote scalability and improve efficiency. Modern ML models can be used to make predictions ranging from outbreaks of disease to the rise and fall of stocks.

Linear Regression Models

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable to be predicted is called the dependent variable. The variable using to predict the other variable's value is called the independent variable.

Simple linear Regression

Simple Linear Regression is a type of Regression algorithms that models the relationship between a dependent variable and a single independent variable. The relationship shown by a Simple Linear Regression model is linear or a sloped straight line, hence it is called Simple Linear Regression.

In Simple Linear Regression *dependent variable must be a continuous/real value*. However, the independent variable can be measured on continuous or categorical values.

Simple Linear regression algorithm has mainly two objectives:

- **Model the relationship between the two variables.** Such as the relationship between Income and expenditure, experience and Salary, etc.
- 1. **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc. Simple Linear Regression: Simple linear regression is a regression technique used to model the relationship between a single input variable (also known as the independent variable or predictor variable) and a continuous output variable (also known as the dependent variable or response variable). It assumes a linear relationship between the input and output variables.

a. Model Representation: In simple linear regression, the relationship between the input variable (x) and the output variable (y) is represented by the equation: $y = \beta_0 + \beta_1 x + \epsilon$

Here,

- y is the output variable (dependent variable),
- x is the input variable (independent variable),
- β_0 is the y-intercept (the value of y when x is 0),
- β_1 is the slope (the change in y for a unit change in x),
- ϵ is the error term or residual (represents the deviation of the actual output from the predicted output).

b. Estimation: The goal of simple linear regression is to estimate the values of β_0 and β_1 that best fit the training data. This is typically done using the method of ordinary least squares (OLS), which minimizes the sum of squared residuals.

OLS estimates the β_0 and β_1 values by minimizing the sum of the squared differences between the observed output values (y) and the predicted values by the linear regression model. The estimated values of β_0 and β_1 are often denoted as b_0 and b_1 , respectively.

c. Model Evaluation: To evaluate the performance of the simple linear regression model, various metrics can be used, including:

- Mean Squared Error (MSE): It calculates the average squared difference between the predicted and actual output values.
- Root Mean Squared Error (RMSE): It is the square root of MSE and provides a more interpretable measure of error.

- Mean Absolute Error (MAE): It measures the average absolute difference between the predicted and actual output values.
 - R-squared (Coefficient of Determination): It represents the proportion of variance in the output variable that can be explained by the input variable. It ranges from 0 to 1, with higher values indicating a better fit.
2. Multiple Linear Regression: Multiple linear regression extends simple linear regression by incorporating multiple input variables to predict a continuous output variable. It models the relationship between the input variables ($x_1, x_2, \dots, x_n$) and the output variable (y) using a linear equation.

a. Model Representation: The relationship between the input variables and the output variable is represented by the equation: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$

Here,

- y is the output variable,
- $x_1, x_2, \dots, x_n$ are the input variables,
- β_0 is the y-intercept,
- $\beta_1, \beta_2, \dots, \beta_n$ are the slopes associated with each input variable,
- ε is the error term.

b. Estimation: Similar to simple linear regression, multiple linear regression estimates the β coefficients using the method of ordinary least squares (OLS) to minimize the sum of squared residuals.

OLS estimates the values of the β coefficients by minimizing the sum of the squared differences between the observed output values (y) and the predicted values by the linear regression model. The estimated values of the β coefficients are often denoted as $b_0, b_1, b_2, \dots, b_n$, respectively.

c. Model Evaluation: Multiple linear regression can be evaluated using the same metrics as simple linear regression, including MSE, RMSE, MAE, and R-squared.

In addition, feature selection techniques can be employed to identify the subset of input variables that contribute significantly to the model's performance. These techniques help in selecting the most relevant features and improving the interpretability of the model.

○ LEAST SQUARES

1. Objective of Least Squares: The objective of the least squares method is to find the parameter values that minimize the sum of the squared residuals. The residuals, also known as errors or residuals, represent the differences between the observed output values and the predicted output values by the linear model. By minimizing the sum of squared residuals, we aim to find the best-fitting line or curve that represents the relationship between the input variables and the output variable.

2. Linear Model Representation: In the context of least squares, we consider a linear model that represents the relationship between the input variables (x) and the output variable (y) as follows:
- $$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Here,

- y represents the output variable,
 - $x_1, x_2, \dots, x_n$ represent the input variables,
 - $\beta_0, \beta_1, \dots, \beta_n$ represent the parameters or coefficients of the linear model,
 - ε represents the error term or residual.
3. Minimization of Residuals: The least squares method aims to minimize the sum of squared residuals (SSE) given by the equation: $SSE = \sum (y - \hat{y})^2$

Here,

- y represents the observed output values,
- $\hat{y}$ represents the predicted output values by the linear model.

By minimizing the SSE, we obtain the parameter values ($\beta_0, \beta_1, \dots, \beta_n$) that result in the best fit of the linear model to the observed data.

4. Ordinary Least Squares (OLS): Ordinary Least Squares (OLS) is a popular approach used to estimate the parameters in linear regression models. It is based on the principle of minimizing the sum of squared residuals.

In OLS, the parameter values ($\beta_0, \beta_1, \dots, \beta_n$) are estimated by solving a system of equations derived from the partial derivatives of the SSE with respect to the parameters. The resulting estimates, denoted as $b_0, b_1, \dots, b_n$, provide the best-fit values for the linear model.

5. Properties of Least Squares Estimators: The least squares estimators obtained through OLS have several desirable properties, including:
- Unbiasedness: The estimators have an expected value equal to the true parameter values in the population.
 - Efficiency: The estimators have minimum variance among all linear unbiased estimators, making them efficient.
 - Consistency: As the sample size increases, the estimators converge to the true parameter values.
 - Normality: Under certain assumptions, the estimators follow a normal distribution.
6. Model Evaluation: After obtaining the parameter estimates using least squares, it is crucial to evaluate the performance of the model. Several metrics can be used, such as:
- Mean Squared Error (MSE): It measures the average squared difference between the observed and predicted values.
 - Root Mean Squared Error (RMSE): It is the square root of the MSE and provides a more interpretable measure of error.
 - Mean Absolute Error (MAE): It measures the average absolute difference between the observed and predicted values.

- R-squared (Coefficient of Determination): It represents the proportion of variance in the output variable explained by the linear model. A higher R-squared value indicates a better fit.

BAYESIAN LINEAR REGRESSION

Bayesian linear regression is an extension of linear regression that incorporates Bayesian inference principles to estimate the parameters of the linear model. It provides a probabilistic framework for modeling and predicting outcomes.

Bayesian Inference: Bayesian inference is a framework for reasoning under uncertainty. It involves updating beliefs about unknown quantities (parameters) based on observed data. The key idea is to assign prior probabilities to the parameters and update them using Bayes' theorem after observing data.

1. **Model Representation:** In Bayesian linear regression, we consider a linear model that represents the relationship between the input variables (x) and the output variable (y) as follows: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$

Here,

- y represents the output variable,
- $x_1, x_2, \dots, x_n$ represent the input variables,
- $\beta_0, \beta_1, \dots, \beta_n$ represent the parameters or coefficients of the linear model,
- ϵ represents the error term or residual.

Bayesian Formulation: Bayesian linear regression incorporates prior distributions over the parameters $\beta_0, \beta_1, \dots, \beta_n$. These prior distributions express our beliefs about the parameter values before observing any data. The choice of priors can influence the model's behavior, and different priors can be used depending on the problem.

$$y \sim N(\beta^T X, \sigma^2 I)$$

3. The output, y is generated from a normal (Gaussian) Distribution characterized by a mean and variance. The mean for linear regression is the transpose of the weight matrix multiplied by the predictor matrix. The variance is the square of the standard deviation σ (multiplied by the Identity matrix because this is a multi-dimensional formulation of the model).

$$P(\beta|y, X) = \frac{P(y|\beta, X) * P(\beta|X)}{P(y|X)}$$

Here, $P(\beta|y, X)$ is the posterior probability distribution of the model parameters given the inputs and outputs. This is equal to the likelihood of the data, $P(y|\beta, X)$, multiplied by the prior probability of the parameters and divided by a normalization constant. This is a simple

expression of Bayes Theorem, the fundamental underpinning of Bayesian Inference. The prior distributions are combined with the likelihood function, which represents the probability of observing the data given the parameter values. In linear regression, the likelihood is often assumed to be normally distributed.

4. **Posterior Distribution:** After observing the data, Bayesian inference updates the prior distributions to obtain the posterior distribution of the parameters. The posterior distribution represents our updated beliefs about the parameter values given the observed data.

$$\text{Posterior} = \frac{\text{Likelihood} * \text{Prior}}{\text{Normalization}}$$

The posterior distribution is proportional to the product of the prior distribution and the likelihood function. It provides a complete distribution over the parameter values, capturing the uncertainty associated with the estimates.

5. **Model Inference:** Given the posterior distribution, various inference tasks can be performed in Bayesian linear regression, including:
 - **Parameter Estimation:** The posterior distribution allows us to estimate the parameter values by calculating the mean, median, or mode of the distribution.
 - **Prediction:** Bayesian linear regression provides a distribution of predictions rather than a single point estimate. The predictive distribution captures the uncertainty in the predictions.
6. **Model Evaluation:** Bayesian linear regression allows for model evaluation using various techniques, including:
 - **Posterior Predictive Checks:** These checks assess whether the model adequately captures the observed data. They involve comparing observed data statistics with statistics generated from the posterior predictive distribution.
 - **Model Comparison:** Bayesian model comparison involves comparing different models using techniques such as Bayes factors or information criteria to determine the model that best fits the data.
7. **Advantages of Bayesian Linear Regression:**
 - **Incorporates uncertainty:** Bayesian linear regression provides a framework for quantifying and propagating uncertainty throughout the modeling process.
 - **Regularization:** Bayesian linear regression naturally incorporates regularization techniques, such as ridge regression or Lasso, by specifying appropriate prior distributions.
 - **Flexibility:** Bayesian linear regression allows for complex model structures, such as hierarchical models or models with varying parameter constraints.

Gradient descent

Gradient descent is an iterative optimization algorithm commonly used in machine learning to minimize the cost function associated with a model. It is a first-order optimization algorithm that updates the model parameters in the direction of steepest descent of the cost function.

Cost Function: In machine learning, we aim to minimize a cost function that quantifies the error or discrepancy between the predicted values of the model and the actual observed values. Let's denote the cost function as $J(\theta)$, where θ represents the model parameters.

1. **Gradient Descent Update:** The goal of gradient descent is to iteratively update the model parameters in the direction of the negative gradient of the cost function. The update equation for a single parameter θ_j is given by: $\theta_j := \theta_j - \alpha * \partial J(\theta) / \partial \theta_j$

Here,

- θ_j represents the j th parameter of the model,
- α is the learning rate (step size) that determines the size of each update,
- $\partial J(\theta) / \partial \theta_j$ represents the partial derivative of the cost function with respect to θ_j .

The learning rate α controls the speed of convergence and the size of the parameter updates. It needs to be carefully chosen to ensure effective learning without overshooting or converging too slowly.

3. **Batch Gradient Descent:** Batch gradient descent computes the gradients using the entire training dataset in each iteration. The update equation for each parameter θ_j can be expressed as: $\theta_j := \theta_j - \alpha * (1/m) * \sum [(h\theta(x(i)) - y(i)) * x(i)_j]$

Here,

- m is the number of training examples,
- $h\theta(x(i))$ represents the predicted value of the model for the i th training example,
- $y(i)$ represents the actual output value of the i th training example,
- $x(i)_j$ represents the j th feature of the i th training example.

Batch gradient descent can be computationally expensive for large datasets as it requires calculating gradients for all training examples in each iteration. However, it guarantees convergence to a global minimum for convex cost functions.

4. **Stochastic Gradient Descent (SGD):** Stochastic gradient descent updates the model parameters using only a single training example at a time. The update equation for each parameter θ_j can be expressed as: $\theta_j := \theta_j - \alpha * (h\theta(x(i)) - y(i)) * x(i)_j$

Here,

- The update is performed after observing each training example $(x(i), y(i))$.

SGD has lower computational requirements per iteration compared to batch gradient descent as it operates on one training example at a time. However, the convergence may be noisier due to the increased variance in the gradients.

5. Mini-Batch Gradient Descent: Mini-batch gradient descent is a compromise between batch gradient descent and SGD. It updates the model parameters using a mini-batch of training examples, where the mini-batch size is typically chosen to be between 10 and 1,000.

The update equation for each parameter θ_j in mini-batch gradient descent is similar to batch gradient descent: $\theta_j := \theta_j - \alpha * (1/\text{batch_size}) * \sum[(h\theta(x(i)) - y(i)) * x(i)_j]$

Here,

- `batch_size` represents the number of training examples in the mini-batch.

Mini-batch gradient descent combines the advantages of both batch and SGD by reducing the computational requirements while maintaining a more stable convergence compared to SGD.

Linear Classification Models: Discriminant function

In linear classification models, the discriminant function is used to determine the class label of an input sample based on its feature values. It maps the feature space to a decision boundary that separates different classes.

Linear Classification: In linear classification, we assume that the decision boundary between classes is a hyperplane in the feature space. The goal is to find the optimal hyperplane that maximally separates the classes. Linear classification models include algorithms such as Perceptron, Logistic Regression, and Support Vector Machines (SVM).

1. Discriminant Function: The discriminant function, denoted as $g(x)$, is a function that takes an input sample x and assigns a class label based on the value of $g(x)$. It is defined as follows: $g(x) = w^T * x + w_0$

Here,

- x represents the feature vector of the input sample,
- w is the weight vector that represents the coefficients of the linear model,
- w_0 is the bias term or threshold.

The discriminant function essentially performs a linear combination of the feature values with the weights and adds the bias term. The resulting value is used to determine the class label of the input sample.

3. Decision Rule: In linear classification, the decision rule is based on the sign of the discriminant function value. If $g(x)$ is positive, the sample is classified as one class, and if $g(x)$ is negative, the sample is classified as the other class. The decision boundary is where $g(x)$ equals zero.
4. Training the Model: To train a linear classification model, we need to find the optimal weight vector w and bias term w_0 . This is typically done by minimizing a cost function that measures the discrepancy between the predicted class labels and the true class labels of the training samples.

Different algorithms use different cost functions and optimization techniques. For example, in logistic regression, the cost function is based on the logistic loss, and the parameters are estimated using maximum likelihood estimation. In SVM, the cost function is based on hinge loss, and the parameters are estimated using convex optimization techniques.

5. Support Vector Machines (SVM): SVM is a widely used linear classification algorithm. In SVM, the discriminant function is defined as: $g(x) = w^T * x + w_0$

The decision boundary is determined by the samples that lie closest to the boundary, called support vectors. SVM aims to maximize the margin between the support vectors of different classes while minimizing the classification errors.

6. Non-Linear Decision Boundaries: While linear classification models assume a linear decision boundary, they can be extended to handle non-linear decision boundaries by applying feature transformations or using kernel functions. These techniques allow for capturing complex relationships between the features and the class labels.

DISCRIMINANT FUNCTION

the discriminant function is mathematically defined as a function that maps the input features to a decision boundary, enabling the classification of input samples into different classes. Let's delve into the mathematical explanation of the discriminant function.

1. Classification Problem: In a classification problem, we have a set of input samples X and their corresponding class labels Y . The goal is to assign class labels to new, unseen samples based on the learned patterns from the training data.
2. Discriminant Function: The discriminant function, denoted as $g(x)$, is a mathematical function that takes an input sample x and produces a real-valued output. It maps the input space to a decision boundary or a region associated with a particular class. The class label assigned to x depends on the value of $g(x)$ and a predefined decision rule.

Mathematically, the discriminant function is represented as: $g(x) = f(x; \theta)$

Here,

- x represents the input sample or feature vector,
- θ denotes the parameters of the discriminant function,
- $f(\cdot)$ represents the functional form or mapping used to compute the discriminant value.

The parameters θ are learned from the training data to optimize the discriminant function for accurate classification.

3. Decision Rule: The decision rule determines how the discriminant function's output is interpreted to assign a class label to the input sample. The decision rule varies depending on the specific classification algorithm being used.

For example, in binary classification, the decision rule can be expressed as: If $g(x) \geq 0$, assign class 1. If $g(x) < 0$, assign class 0.

The specific decision rule may differ for multiclass classification problems, but the underlying principle remains the same—determining the class label based on the discriminant function's output.

4. **Learning the Discriminant Function:** To learn the discriminant function, various machine learning algorithms employ different approaches. These algorithms aim to find the optimal values for the parameters θ that minimize a specified objective function or maximize a discriminative criterion.

For instance, in logistic regression, the discriminant function is obtained by applying the sigmoid function to a linear combination of the input features: $g(x) = \sigma(w^T * x)$

Here, $\sigma(\cdot)$ represents the sigmoid function, and w is the weight vector.

In support vector machines (SVM), the discriminant function is defined as a linear combination of support vectors: $g(x) = \sum(\alpha_i * y_i * K(x, x_i)) + b$

Here,

- α_i and y_i are parameters obtained from the SVM optimization process,
- $K(\cdot, \cdot)$ represents the kernel function,
- x_i represents the support vectors,
- b denotes the bias term.

Different algorithms may employ different functional forms and optimization techniques to learn the discriminant function.

Probabilistic discriminative model

a probabilistic discriminative model refers to a classification model that directly models the posterior probability of class labels given the input features. It aims to estimate the conditional probability distribution $P(Y|X)$, where Y represents the class labels and X represents the input features. Let's explore the mathematical explanation of a probabilistic discriminative model.

1. **Notation:**
 - X : The input feature vector or sample.
 - Y : The class label associated with the input sample.
 - $P(Y|X)$: The conditional probability distribution of the class labels given the input features.
2. **Probabilistic Discriminative Model:** A probabilistic discriminative model directly models the posterior probability distribution $P(Y|X)$. It captures the relationship between the input features and the class labels and estimates the probability of each class label given the observed features.

Mathematically, a probabilistic discriminative model can be represented as: $P(Y|X) = f(X; \theta)$

Here,

- $f(\cdot)$ denotes the functional form or mapping used to estimate the posterior probability distribution.
- θ represents the parameters of the model that need to be learned from the training data.

The goal is to estimate the optimal parameter values θ that maximize the likelihood of the observed training data.

3. **Parameter Estimation:** The parameter estimation process in probabilistic discriminative models involves finding the optimal values of θ that maximize the likelihood of the observed training data. This is typically done using techniques such as maximum likelihood estimation or Bayesian inference.

Maximum Likelihood Estimation (MLE): In MLE, the parameters θ are estimated by maximizing the likelihood function of the training data. The likelihood function measures the probability of observing the given training data given the parameter values. The optimal parameter values are obtained by solving the following optimization problem: $\theta^* = \operatorname{argmax}_{\theta} [\prod P(Y_i|X_i; \theta)]$

Bayesian Inference: In Bayesian inference, a prior distribution is defined over the parameters θ , and the posterior distribution is computed by combining the prior distribution with the likelihood function. The optimal parameter values are obtained by finding the posterior distribution that maximizes the posterior probability given the observed data.

4. **Decision Rule:** Once the posterior probability distribution $P(Y|X)$ is estimated, the decision rule is applied to assign a class label to a new, unseen sample. The decision rule can be based on the maximum a posteriori (MAP) estimation or other criteria, depending on the specific classification problem.

For example, in binary classification, the decision rule can be expressed as: If $P(Y=1|X) > P(Y=0|X)$, assign class 1. If $P(Y=1|X) < P(Y=0|X)$, assign class 0.

5. **Examples of Probabilistic Discriminative Models:**
 - **Logistic Regression:** In logistic regression, the posterior probability is estimated using the logistic function, which models the probability of the positive class label.
 - **Gaussian Discriminant Analysis (GDA):** GDA assumes that the conditional probability distribution of the features given the class label follows a Gaussian distribution. It estimates the posterior probability using Bayes' theorem.

LOGISTIC REGRESSION

logistic regression is a widely used probabilistic discriminative model for binary classification. It models the posterior probability of the positive class label using a logistic function. Let's delve into the mathematical explanation of logistic regression.

1. **Binary Classification Problem:** In binary classification, we have a set of input samples X and their corresponding class labels Y , where Y belongs to $\{0, 1\}$. The goal is to estimate the probability of the positive class label given the input features.
2. **Logistic Regression Model:** In logistic regression, the model assumes that the posterior probability of the positive class label $P(Y=1|X)$ can be modeled using a logistic or sigmoid function.

Mathematically, the logistic regression model is defined as: $P(Y=1|X) = \sigma(w^T * X + b)$

Here,

- $P(Y=1|X)$ represents the probability of the positive class label given the input features X .
- $\sigma(\cdot)$ denotes the logistic or sigmoid function, defined as $\sigma(z) = 1 / (1 + \exp(-z))$.
- w represents the weight vector.
- b represents the bias term.

The logistic regression model performs a linear combination of the input features X with the weight vector w , adds the bias term b , and passes the result through the sigmoid function to obtain the estimated probability.

3. **Parameter Estimation:** The parameters w and b of the logistic regression model need to be estimated from the training data. This is typically done by maximizing the likelihood of the observed training data.

The likelihood function for logistic regression is defined as the product of the conditional probabilities for each sample. Assuming the samples are independent and identically distributed, the likelihood function can be expressed as: $L(w, b) = \prod [P(Y_i=1|X_i; w, b)]^{Y_i} * [P(Y_i=0|X_i; w, b)]^{(1 - Y_i)}$

The optimal parameter values w^* and b^* can be obtained by maximizing the likelihood function.

4. **Optimization:** Maximizing the likelihood function is typically done by minimizing the negative log-likelihood, which is equivalent to minimizing the cross-entropy loss function. The negative log-likelihood function for logistic regression is given by: $J(w, b) = - \sum [Y_i * \log(P(Y_i=1|X_i; w, b)) + (1 - Y_i) * \log(P(Y_i=0|X_i; w, b))]$

The objective is to find the values of w and b that minimize the loss function $J(w, b)$. This can be done using optimization algorithms such as gradient descent or Newton's method.

5. **Decision Rule:** Once the logistic regression model is trained and the optimal parameter values are obtained, the decision rule is applied to classify new, unseen samples. The decision rule is typically based on a threshold value, such as 0.5. If the estimated probability $P(Y=1|X)$ is above the threshold, the sample is classified as the positive class; otherwise, it is classified as the negative class.

6. Regularization: To prevent overfitting, logistic regression can incorporate regularization techniques such as L1 or L2 regularization. This adds a penalty term to the loss function to control the complexity of the model and avoid extreme parameter values.

Probabilistic generative model

a probabilistic generative model is a type of model that learns the joint probability distribution of the input features and class labels. It estimates the class-conditional probability distributions and the prior probabilities of the class labels. Let's explore the mathematical explanation of a probabilistic generative model.

1. Notation:
 - X: The input feature vector or sample.
 - Y: The class label associated with the input sample.
 - $P(X)$: The probability distribution of the input features.
 - $P(Y)$: The prior probability distribution of the class labels.
 - $P(X|Y)$: The class-conditional probability distribution of the input features given the class labels.
 - $P(Y|X)$: The posterior probability distribution of the class labels given the input features.
2. Probabilistic Generative Model: A probabilistic generative model aims to learn the joint probability distribution $P(X, Y)$ of the input features and class labels. It models the class-conditional distributions $P(X|Y)$ and the prior probability distribution $P(Y)$.

Mathematically, a probabilistic generative model can be represented as: $P(X, Y) = P(X|Y) * P(Y)$

The goal is to estimate the parameters of the class-conditional distributions and the prior probabilities from the training data.

3. Parameter Estimation: The parameter estimation process in probabilistic generative models involves estimating the parameters of the class-conditional distributions and the prior probabilities.

For example, in Gaussian Naive Bayes, a commonly used probabilistic generative model, the class-conditional distributions are assumed to follow Gaussian distributions. The mean and covariance matrix for each class can be estimated from the training data. The prior probabilities can be estimated by counting the relative frequencies of each class label in the training data.

4. Bayesian Inference: Probabilistic generative models enable Bayesian inference, which involves computing the posterior probability distribution $P(Y|X)$ given the observed data.

Using Bayes' theorem, the posterior probability can be calculated as: $P(Y|X) = P(X|Y) * P(Y) / P(X)$

Here,

- $P(Y|X)$ represents the posterior probability of the class labels given the input features.
- $P(X|Y)$ denotes the class-conditional probability distribution.
- $P(Y)$ represents the prior probability distribution.

- $P(X)$ is the marginal probability distribution of the input features.

In practice, the denominator $P(X)$ can be ignored when comparing posterior probabilities since it acts as a normalization constant.

5. **Decision Rule:** Once the posterior probability distribution $P(Y|X)$ is estimated, the decision rule is applied to assign a class label to a new, unseen sample. The decision rule can be based on the maximum a posteriori (MAP) estimation or other criteria, depending on the specific classification problem.

For example, in binary classification, the decision rule can be expressed as: If $P(Y=1|X) > P(Y=0|X)$, assign class 1. If $P(Y=1|X) < P(Y=0|X)$, assign class 0.

6. **Examples of Probabilistic Generative Models:**

- **Gaussian Naive Bayes:** It assumes that the class-conditional distributions follow Gaussian distributions with independent features.
- **Gaussian Mixture Models (GMM):** GMM represents the class-conditional distributions as a mixture of Gaussian distributions.
- **Hidden Markov Models (HMM):** HMM models sequential data with hidden states and generates class labels based on observed data.

Naive Bayes

1. **Bayes' Theorem:** Bayes' theorem is a fundamental concept in probability theory that allows us to update the probability of an event based on new evidence. It is given as:

$$P(Y|X) = P(X|Y) * P(Y) / P(X)$$

Here,

- $P(Y|X)$ is the posterior probability of class Y given the input features X .
 - $P(X|Y)$ is the likelihood of observing the input features X given class Y .
 - $P(Y)$ is the prior probability of class Y .
 - $P(X)$ is the marginal probability of the input features.
2. **Naive Bayes Classifier:** The Naive Bayes classifier makes a naive assumption that the features are conditionally independent given the class label Y . This simplifies the calculation of the likelihood term $P(X|Y)$ in Bayes' theorem.

Mathematically, the Naive Bayes classifier can be represented as:

$$P(Y|X) = P(Y) * P(X_1|Y) * P(X_2|Y) * \dots * P(X_n|Y)$$

Here,

- $P(Y|X)$ is the posterior probability of class Y given the input features X .
- $P(Y)$ is the prior probability of class Y .
- $P(X_i|Y)$ is the conditional probability of feature X_i given class Y .

The Naive Bayes classifier assumes that each feature X_i is conditionally independent of other features given the class label Y . This assumption allows us to factorize the likelihood term as a product of individual conditional probabilities.

3. **Parameter Estimation:** To apply the Naive Bayes classifier, we need to estimate the parameters involved: the prior probabilities $P(Y)$ and the conditional probabilities $P(X_i|Y)$.
 - **Prior Probability:** The prior probability $P(Y)$ can be estimated by counting the relative frequencies of each class label in the training data.
 - **Conditional Probability:** The conditional probabilities $P(X_i|Y)$ can be estimated differently based on the type of features:
 - a) **Categorical Features:** For categorical features, the conditional probability $P(X_i|Y)$ can be estimated by counting the relative frequencies of each feature value given the class label.
 - b) **Continuous Features:** For continuous features, the conditional probability $P(X_i|Y)$ can be estimated using probability density functions such as Gaussian distribution. The parameters of the distribution (e.g., mean and variance) can be estimated from the training data.
4. **Decision Rule:** To classify a new, unseen sample, the Naive Bayes classifier applies a decision rule based on the posterior probabilities. The class label with the highest posterior probability is assigned to the sample.

Mathematically, the decision rule can be expressed as: Predicted Class = $\text{argmax}_y P(Y=y) * P(X|Y=y)$

Here,

- argmax_y represents the class label that maximizes the expression.
 - $P(Y=y)$ is the prior probability of class y .
 - $P(X|Y=y)$ is the likelihood term for class y .
5. **Diagram of Naive Bayes Classifier:** A diagram of the Naive Bayes classifier is typically represented as a directed acyclic graph (DAG) that illustrates the conditional dependencies among the features and the class label. However, in the Naive Bayes classifier, the assumption of feature independence implies that there are no direct dependencies between the features. Therefore, the diagram for the Naive Bayes classifier would simply consist of arrows from the class label to each feature, indicating the conditional dependence.

MAXIMUM MARGIN CLASSIFIER

The Maximal-Margin Classifier is a hypothetical classifier that best explains how SVM works in practice.

The numeric input variables (x) in your data (the columns) form an n -dimensional space. For example, if you had two input variables, this would form a two-dimensional space.

A hyperplane is a line that splits the input variable space. In SVM, a hyperplane is selected to best separate the points in the input variable space by their class, either class 0 or class 1. In two-dimensions you can visualize this as a line and let's assume that all of our input points can be completely separated by this line. For example:

$$B_0 + (B_1 * X_1) + (B_2 * X_2) = 0$$

Where the coefficients (B_1 and B_2) that determine the slope of the line and the intercept (B_0) are found by the learning algorithm, and X_1 and X_2 are the two input variables.

You can make classifications using this line. By plugging in input values into the line equation, you can calculate whether a new point is above or below the line.

- Above the line, the equation returns a value greater than 0 and the point belongs to the first class (class 0).
- Below the line, the equation returns a value less than 0 and the point belongs to the second class (class 1).
- A value close to the line returns a value close to zero and the point may be difficult to classify.
- If the magnitude of the value is large, the model may have more confidence in the prediction.

The distance between the line and the closest data points is referred to as the margin. The best or optimal line that can separate the two classes is the line that has the largest margin. This is called the Maximal-Margin hyperplane.

The margin is calculated as the perpendicular distance from the line to only the closest points. Only these points are relevant in defining the line and in the construction of the classifier. These points are called the support vectors. They support or define the hyperplane.

The hyperplane is learned from training data using an optimization procedure that maximizes the margin.

Soft Margin Classifier

In practice, real data is messy and cannot be separated perfectly with a hyperplane.

The constraint of maximizing the margin of the line that separates the classes must be relaxed. This is often called the soft margin classifier. This change allows some points in the training data to violate the separating line.

An additional set of coefficients are introduced that give the margin wiggle room in each dimension. These coefficients are sometimes called slack variables. This increases the complexity of the model as there are more parameters for the model to fit to the data to provide this complexity.

A tuning parameter is introduced called simply C that defines the magnitude of the wiggle allowed across all dimensions. The C parameter defines the amount of violation of the margin allowed. A $C=0$ is no violation and we are back to the inflexible Maximal-Margin Classifier described above. The larger the value of C the more violations of the hyperplane are permitted.

During the learning of the hyperplane from data, all training instances that lie within the distance of the margin will affect the placement of the hyperplane and are referred to as support vectors. And as C affects the number of instances that are allowed to fall within the margin, C influences the number of support vectors used by the model.

- The smaller the value of C , the more sensitive the algorithm is to the training data (higher variance and lower bias).
- The larger the value of C , the less sensitive the algorithm is to the training data (lower variance and higher bias).

Support vector machine,

support vector machine (SVM), generalized under the name *kernel machine*, has

been popular in recent years for a number of reasons:

1. It is a discriminant-based method and uses Vapnik's principle to never solve a more complex problem as a first step before the actual problem
 2. After training, the parameter of the linear model, the weight vector, can be written down in terms of a subset of the training set, which are the so-called *support vectors*. In classification, these are the cases that are close to the boundary and as such, knowing them allows knowledge extraction
 3. The output is written as a sum of the influences of support vectors and these are given by *kernel functions* that are application-specific measures of similarity between data instances.
 4. Typically in most learning algorithms, data points are represented as vectors, and either dot product (as in the multilayer perceptrons) or Euclidean distance (as in radial basis function networks) is used. A kernel function allows us to go beyond that.
 5. Kernel-based algorithms are formulated as convex optimization problems, and there is a single optimum that we can solve for analytically. Therefore we are no longer bothered with heuristics for learning rates, initializations, checking for convergence, and such.
1. Linear SVM: Assume we have a training dataset with labeled samples, where each sample is represented by a feature vector x and belongs to either class +1 or class -1.

The goal of a linear SVM is to find a hyperplane that separates the two classes in the feature space. The hyperplane is defined by a weight vector w and a bias term b , and it can be represented as:

$$w^T x + b = 0$$

Here, w^T denotes the transpose of the weight vector.

2. Decision Function: Given a new input sample x , the decision function of the linear SVM is defined as:

$$f(x) = \text{sign}(w^T x + b)$$

If $f(x)$ is positive (+1), the sample is classified as class +1, and if $f(x)$ is negative (-1), the sample is classified as class -1

3. Margin: The margin is the distance between the decision boundary (hyperplane) and the closest samples from each class. To maximize the margin, SVMs aim to find the optimal weight vector w and bias term b .

Let's consider two parallel hyperplanes that pass through the support vectors, one for each class. The margin is the distance between these two hyperplanes, and it can be computed as:

$$\text{margin} = (w^T x_+ + b) - (w^T x_- + b) / \|w\|$$

Here, x_+ and x_- are the support vectors from each class, and $\|w\|$ represents the Euclidean norm of the weight vector.

4. Optimization: To find the optimal weight vector and bias term, SVMs formulate an optimization problem. The objective is to maximize the margin while minimizing the classification errors.

Mathematically, this can be formulated as a constrained optimization problem:

$$\begin{aligned} &\textbf{Minimize: } (1/2) * \|w\|^2 \\ &\textbf{Subject to: } y_i * (w^T x_i + b) \geq 1, \text{ for all training samples } (x_i, y_i) \end{aligned}$$

Here, y_i is the class label (+1 or -1) of the training sample x_i .

5. Dual Form: By introducing Lagrange multipliers α_i , the optimization problem can be transformed into its dual form. The dual problem involves maximizing a function with respect to the Lagrange multipliers, subject to certain constraints.

The dual problem is given by:

$$\begin{aligned} &\textbf{Maximize: } \sum \alpha_i - (1/2) * \sum \sum \alpha_i * \alpha_j * y_i * y_j * x_i^T x_j \\ &\textbf{Subject to: } \sum \alpha_i * y_i = 0 \\ &\quad \alpha_i \geq 0, \text{ for all } i \end{aligned}$$

Here, α_i are the Lagrange multipliers, and x_i and x_j are the training samples.

6. Classification: Once the Lagrange multipliers α_i are obtained, the decision function for classifying new samples becomes:
 $f(x) = \text{sign}(\sum \alpha_i * y_i * x_i^T x + b)$

Here, x_i and y_i correspond to the support vectors with non-zero Lagrange multipliers.

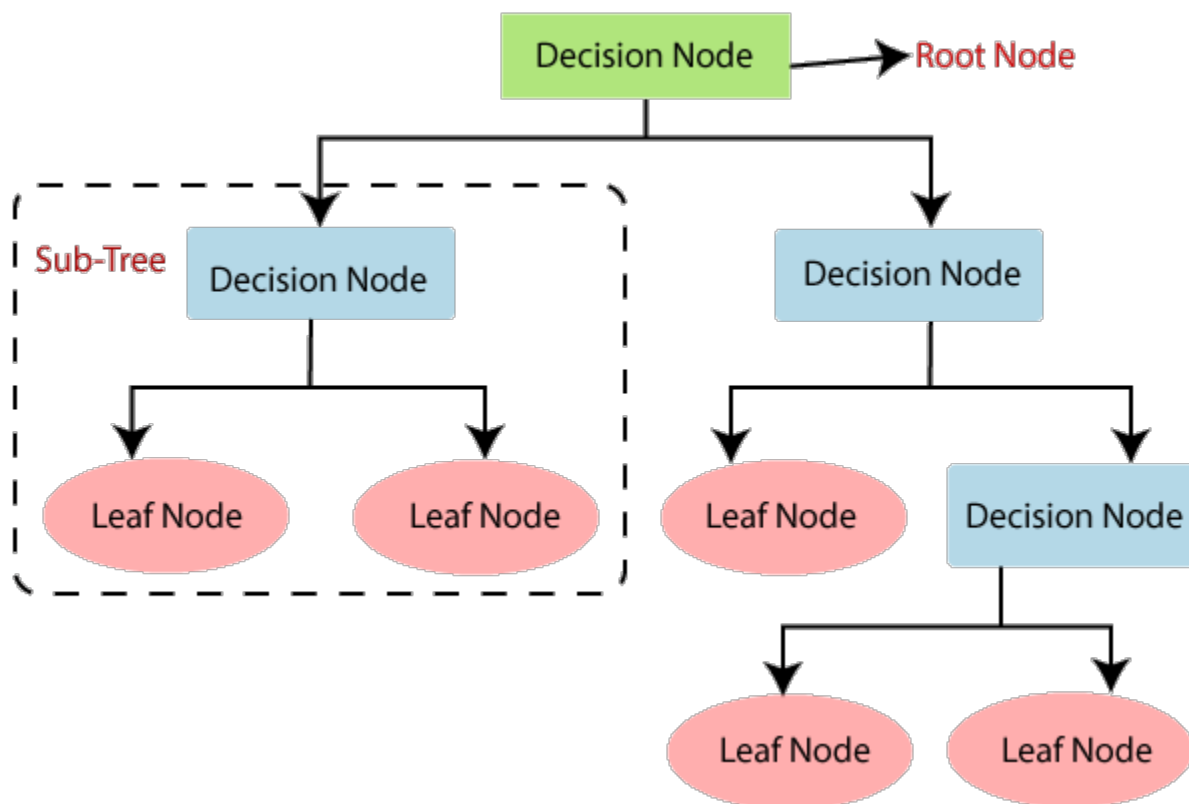
These are the mathematical principles behind linear SVMs for binary classification

Decision Tree,

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.**
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches,

whereas Leaf nodes are the output of those decisions and do not contain any further branches.

- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further splits the tree into subtrees.
- Below diagram explains the general structure of a decision tree:



Why use Decision Trees?

there are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model.

Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

🎬 **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

🎬 **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

🎬 **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

🎬 **Branch/Sub Tree:** A tree formed by splitting the tree.

🎬 **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

🎬 **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

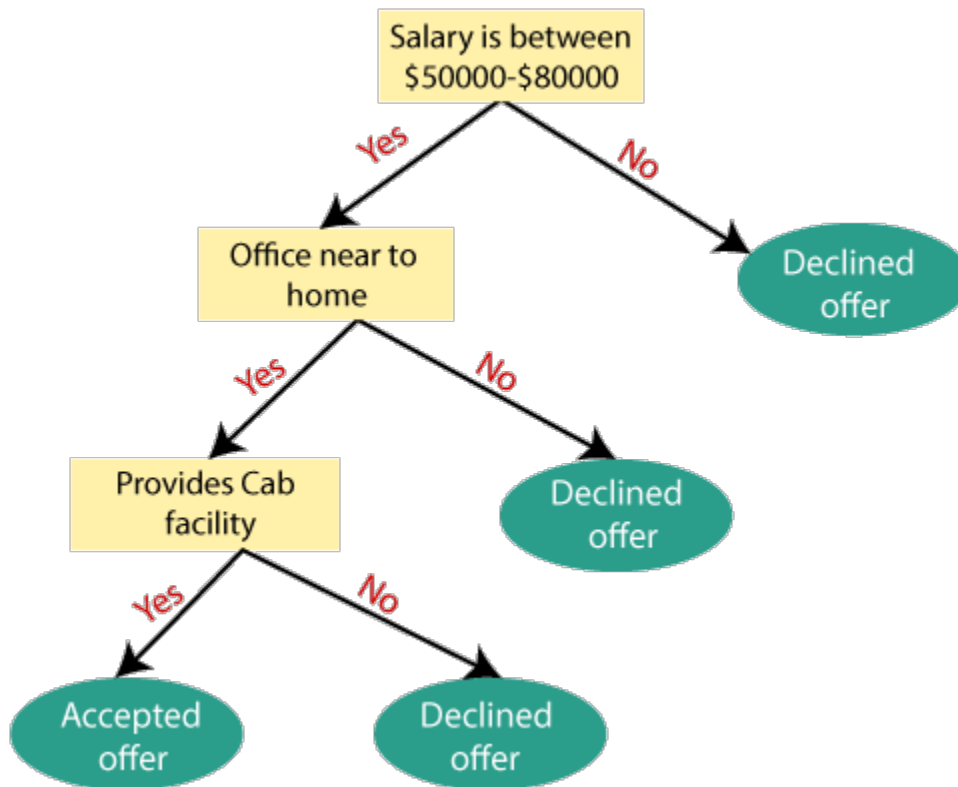
Decision Tree algorithm

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node. For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root

node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- **Information Gain**
- **Gini Index**

1. Information Gain:

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.

- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

$$1. \text{ Information Gain} = \text{Entropy}(S) - [(\text{Weighted Avg}) * \text{Entropy}(\text{each feature})]$$

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes}) \log_2 P(\text{yes}) - P(\text{no}) \log_2 P(\text{no})$$

Where,

- **S= Total number of samples**
- **P(yes)= probability of yes**
- **P(no)= probability of no**

2. Gini Index:

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\text{Gini Index} = 1 - \sum_j P_j^2$$

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- **Cost Complexity Pruning**
- **Reduced Error Pruning.**

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

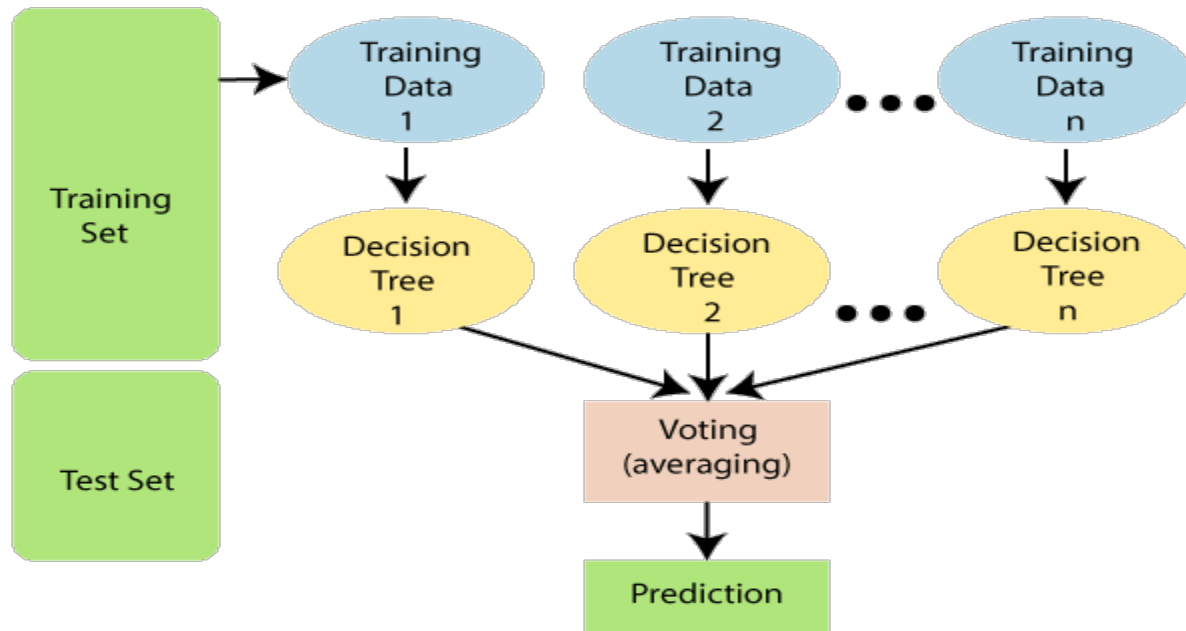
Random forests

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model*.

As the name suggests, *"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."* Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

The below diagram explains the working of the Random Forest algorithm:



Assumptions for Random Forest

Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not. But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random forest classifier:

- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
- The predictions from each tree must have very low correlations.

Random Forest algorithm

Random Forest works in two-phase first is to create the random forest by combining N decision tree, and second is to make predictions for each tree created in the first phase.

The Working process can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

Step-2: Build the decision trees associated with the selected data points (Subsets).

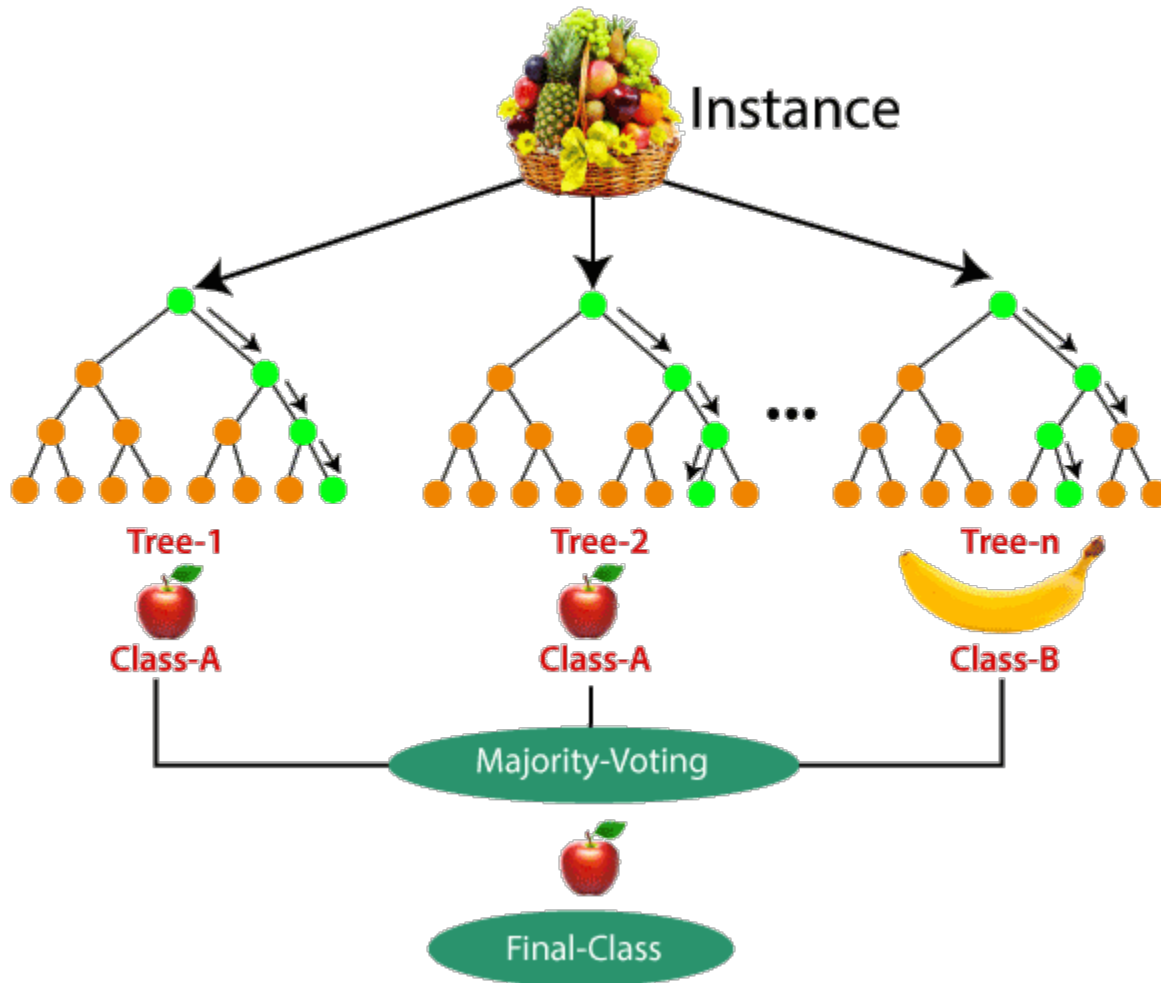
Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

The working of the algorithm can be better understood by the below example:

Example: Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision. Consider the below image:



Applications of Random Forest

There are mainly four sectors where Random forest mostly used:

1. **Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
2. **Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
3. **Land Use:** We can identify the areas of similar land use by this algorithm.
4. **Marketing:** Marketing trends can be identified using this algorithm.

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.